MODULE-1.

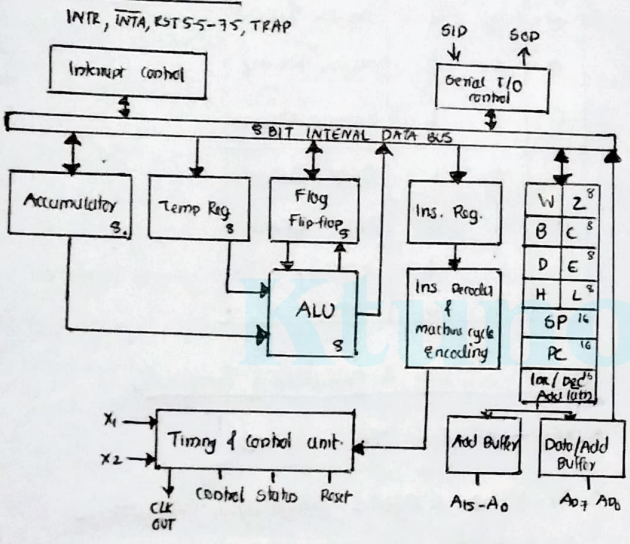
MP

- programmable device that works in no's, performs on them arithmatic or logical operations according to the pgm stored in memory and then produces other no's as a result.

Features

1. 8-bit
2. NMOS chip
3. 40 pin
4. LSI chip
5. +5V
6. 3 MHz clk
7. 80 inst.
8. 246 Opcode

Architecture of 8085



1. ALU
   - → It perform arithmatic and logical opera.
   - → add, sub, AND, OR etc on 8-bit data.

2. Accumulator
   - → 8-bit register - part of ALU
   - → It store 8-bit data & to perform arithmatic & logical opera.
   - → Result of oper → stored in Accumulator.

3. General purpose Registers
   - → 6 - GPR. → B, C, D, E, H & L.
   - → Each can hold 8-bit data
   - → can work in pair to hold 16-bit → BC, DE, & HL
   - → Programer can copy or store data.

4. Temporary Register
   - → 8-bit register - Associated with ALU
   - → It hold data during Arith/logical opera

5. Flag Register
   - → 8-bit register - 5-FF
   - → Array of FF - gives status of the opera.

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	X	AC	X	P	X	CY

Sign, zero, Auxiliary carry, parity, carry.

6. Instruction Register and decoder
   - → 8-bit register
   - → It hold op code of ins which is being to be executed.
   - → Ins. dec. decodes the information present in inst. register.

7. Program counter
   - → 16-bit
   - → used to store the memory add. location of the nxt. inst. to be fetched.
   - → MP INC → PC → whenever an inst. executed.

8. Stack pointer
   - → 16-bit.
   - → It points to the top element of data stored in the stack.

9. Address buffer & address-data buffer
   - → content of SP & PC is loaded → AB & ADB to communicate with CPU.

10. Address & Data bus

    Address Bus → carries the location to where it should be stored &
    → Unidirectional

    Data Bus → carries data to be stored
    → Bidirectional

11. Timing & control unit
    - → generate & provide timing & control signals to MP for execution of opera.
    - CPU & peripheral
    - opera. of MP & peripheral

Control signals : READY, RD, WR, ALE

Status signals : S0, S1, IO/M

DMA signals: HOLD, HLDA

Reset signals: RESET IN, RESET OUT

12. Interrupt control

→ It controls interrupts during a process

→ 5 interrupt signals

INTR, ~~INT~~ RST 7.5 → 5.5 & TRAP.

13. Serial I/O control

→ It controls serial data communication

→ SID & SOD

Pinout classification (6 groups)

1. Address Bus
2. Data Bus
3. Control and status signals.
4. Power supply & frequency signals.
5. Externally initiated signals
6. Serial I/O ports.

1. Address & Data bus

→ 16 signal lines – used as the addl. bus

→ Split into 2 segments → A_{15} – A_8 → unidirection
High order address of 16 bit

AD_7 – AD_0 → Bidirection
Low order address of 16 bit

2. Control & Status signals

Control signals: High → address
Low → Data ↓

1. ALE :→ Indicate a valid address on the bus
→ positive going pulse used to latch the address on the bus.

2. $\overline{RD}$: Read memory / IO device.
→ Indicate selected memory / IO device is to be read
→ Data bus is ready for accepting data from the M/IO device

3. WR : Write memory / IO

→ Indicate that data on the data bus is to be written into selected memory location / IO device

Status signals

1. ~~IO/M~~ IO/$\overline{M}$ – Select memory or I/O device
when = 1 ⇒ I/O operation
= 0 ⇒ $\overline{M}$ memory operation

2. S1, S0

IO/$\overline{M}$	S_1	S_0	OPERATION.
0	0	1	Mem. WRITE
0	1	0	Mem. READ
0	1	1	OPCODE FETCH
1	0	1	I/O WRITE
1	1	0	I/O READ
1	1	1	INTA.
	0	0	HLT.

3. Power supply & frequency signals

- V_{CC} – +5V V_{SS} – GND
- X_1, X_2 ⇒ crystal oscillator connections to set frequency of internal CLK generator
- CLK (out) ⇒ Clock output is used as s/m clock for peripheral device connected to MP

4. Externally Initiated signals.

1. DMA → Direct memory Access
– used for large volu. data transfer b/w memory & I/O device directly.

2. HOLD → To request the control of s/m bus.

3. HLDA → on the receive of HOLD sig. the MP acknowledges the request by sending out HLDA.
→ After getting HLDA sig. the DMA starts direct transfer of data.

4. READY: I/P. It indicates whether the memory or IO device ready for data trans.

5. RESET IN: used to reset the MP. PC → 0.

6. RESET OUT: Reset all the connected device when MP is reset

5. Serial i/p control signals

1. SID → Serial i/p data line
2. SOD → Serial o/p data line

Interrupt control signals

- Generated by external device to request the MP to perform a task.
- 5 interrupt signals (H/W).

1. TRAP
2. RST 7.5
3. RST 6.5
4. RST 5.5
5. INTR.

INTA – Aknowledgement signal (interrupt)

Instruction

Opcode → gives information about what to do

Operand → where to do

1byte = 8 bit

Classification (Based on word size)

1. One byte inst.

→ It includes opcode & operand in same byte.

Eg: MOV A,C , ADD B

2. Two byte inst

→ 1st byte → opcode ;
2nd byte → operand.

Ex: MVI A, 08

3. Three byte inst

→ 1st byte → Opcode
2nd & 3rd → 16-bit address

Ex: LDA 2050. , LXI

Addressing Modes

1. Direct add. mode
2. Register add. mode
3. Register Indirect add. mode
4. Immediate add. mode
5. Implicit add. mode

1. Direct add.

→ Address of data is given in the inst. itself
→ 3 byte inst.
→ LDA 2000 , STA 2400

2. Register add.

→ Data is in one of the registers
→ MOV A,B , MOV A,B

3. Register Indirect add.

→ The data is in memory, the add. of memory is not directly given in the inst
→ MOV A,M – move the content of the memory to accumulator, whose add. is in the HL pair.

4. Immediate add.

→ operand is specified with in the inst
→ MVI A, 08
→ data is placed immediately after the opcode

5. Implicit add.

→ Inst. which operates on the content of the accumulator.
→ CMP, RLC,

Instruction set:

1. Data transfer Inst

- It performs transfer of data from one register to another, from mem → reg. or reg → mem.

Ex: MOV , MVI , LXI

- 1 byte inst

- It perform arithmetic opera. of the content of a register or memory

Ex: ADD, SUB, INR, DEC

3. Logical group
- It perform logical operations
- ANA, XRA, ORA, CMP, RAL

4. Branch control group - Branching inst
- Instructions for conditional & unconditional jump subroutine, CALL & RET & Restart.

Ex. JMP, JC, JZ, CALL
control EI EA

5. I/O & machine control - control inst
- Inst which control the I/O & machine.
- IN $\overset{\text{port}}{\wedge}$ - I/p data to accu from a port with 8-bit add.
- OUT port - A → o/p data
- HLT - Stop executing pgm
- NOP - No operation

DI - disable interrupt
EI - Enable interrupt

Instruction cycle:

1. Machine cycle:
- It is the time required to complete one operation of accessing the memory / I/O devices.
- consist 3 - 6 T states
- Every machine cycle, first ope. is op-code fetch
- op code fetch, mem Read, mem write, I/O Read, I/O write, INTR Acknowledge, Bus idle.

2. Instruction cycle
- Time required to execute an instruction

3. T-state
- It is one time corresponding to one clock period
- Basic unit to calc the time taken for each execution of instr. & program in a processor.

$T = \frac{1}{f}$ f - internal clk freq

Timing Diagram:
- It is a graphical representation
- represents the execution time taken by each inst in a graphical format.

8085 MP has following machine cycle.
1. Opcode fetch - (4T or 6T)
2. Mem. Read - 3T
3. Mem write - 3T
4. I/O Read - 3T
5. I/O write - 3T

Following buses & control signals must be shown in a timing diagram
1. Higher order add. $\overset{\text{bus}}{\wedge}$ - $A_8 - A_{15}$
2. Multiplexed Add/Bus - $AD_0 AD_7$
3. ALE
4. $IO/\overline{M}$
5. S_0 S_1
6. $\overline{RD}$
7. $\overline{WR}$

- A pgm is a set of logically related instructions written in a specific sequence to accomplish a task

- Compiler - reads the entire pgm first & translate it into the object code

- Interpreter - reads one inst at a time & produces its object code & execute. the inst. before reading next inst

Steps to write a pgm

1. Analyze the pgm problem
2. Develop pgm logic
3. Write algorithm
4. Make a flowchart
5. Write program instru. using ALP

Multiply two number

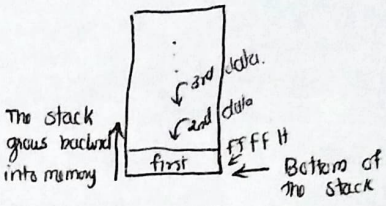
```
LDA  2000 H
MOV B, A
LDA  2001 H
MOV C, A
MVI A, 00 H
L1: ADD B
DCR C
JNZ L1
STA 2010
HLT
```

The stack.

- It is an area of memory identified by the programmer for temporary storage of information

- It works in LIFO manner.

- The stack normally grows backwards into memory



- Stack is shared by pgmr & MP

- LXI SP, FFFF H → Setting SP to location FFFF H (end of the mem for boss)

Saving data

- Information is saved by PUSH inst

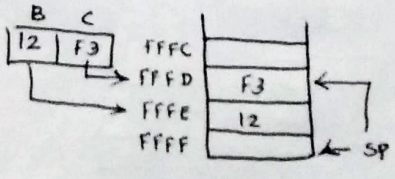
- It is retrieved back by POP inst.

- Both PUSH & POP work with register pairs only.

PUSH instruction (1 Byte inst)

~~- 1 Byte inst.~~ steps ↓

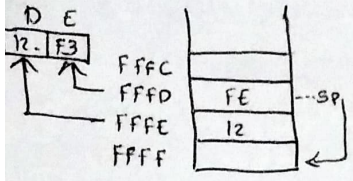
- Decrement SP
- copy the content of reg B to the memory location pointed to by SP.
- Decrement SP
- copy the content of reg C to the memo. location pointed to by SP



POP instruction (1 Byte inst)

- copy the contents of mem. location pointed by SP to reg E
- Increment SP
- loca. pointed by SP to reg D.

- Increment SP.



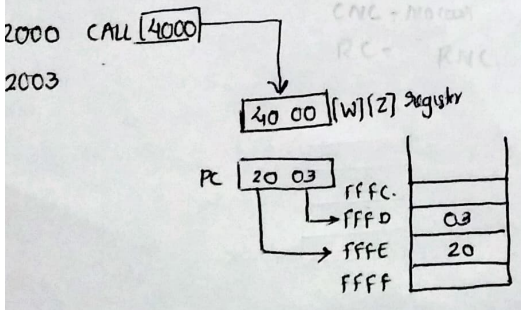
Subroutines

- group of inst that will be used separately in different locations of the pgm.
- Rather than repeat the same inst several times they can be grouped into a subroutine that is called from diff. location
- In ALP, a subroutine can exist anywhere in the code.
- The subroutines ~~must~~ mainly placed ~~sp~~ separately from the main pgm.
- 8085 has 2 inst.
  1. CALL → Redirect pgm execution to the subroutine
  2. RET → Return the execution to the calling routine

The CALL inst

. CALL 4000H (3 byte inst)

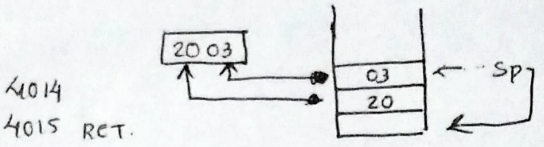
- When CALL inst is fetched, the MP knows that next two location contain 16-bit subroutine addr. in the memory



- MP reads the subroutine pgm from the next two memory location and store the HO-8-bit of the addr. in the [W] regis & stors LO-8-bit of the addr. → [Z] regis
- pushes the current value of PC (addr)
- loads the PC with 16-bit address supplied with the CALL inst. from [W][Z] register

RET (1byte)

- Retrieve the return addr. from top of SP.
- loads the PC with the return add.



DELAY ROUTINE

- They are subroutines used for maintaining the timings of various operations in MP

Delay routine process

- written as subroutine. → a count is loaded in reg.
→ It DEC by 1 & zero flag is checked → To ver fy zero or not. → continued untill reg is zero.
→ when it is zero → time delay is over & return back to main pgm to carry out desired operation

Delay:

- Total time taken to execute the delay routine

Q: write a delay routine to produce a time delay of 0.5 ms in 8085 - clk f = 3 MHz.

→ Delay required = 0.5 ms.

```
    MVI D, N
L1: DCR D
    JNZ L1
    RET
```

RUN + 32

T for 1 T state = $\frac{1}{f}$

$= \frac{1}{3 \times 10^6}$

$= 0.33 \mu s$

No. T-state required for delay of 0.5 ms = $\frac{\text{Required time delay}}{\text{Time for one T-state}}$

$\frac{0.5 \text{ ms}}{0.333} = 1500.10$

$K(N+32) = 1500$

$N = 104.857 = 105_{10} = 69_H$

---

2. Write an ALP for 8085 to count from AA_H to 00_H, with time delay of 2ms for each count. f = 1MHz.

$T_{Total} = \frac{1}{1 \times 10^6} = 1\mu s$

Main pgm.

```
MVI C, AAH
L1: CALL DELAY
    DCR C
    JNZ L1
    HLT.
```

Sec Delay pgm

```
DELAY: MVI D, 4AH
Next: NOP
      NOP
      NOP
      NOP
      DCR D
      JNZ Next
      RET
```

---

1. Addition of 2 8-bit

① 19H + 56H

[2501] ← 19 [2502] ← 56 [2503] → 75.

```
LDA 2501
MOV B,A
LDA 2502
ADD B
STA 2503
HLT.
```

---

2. Addition of 2 8-b with carry

```
MVI C,00
LDA 4150
MOV B,A
LDA 4151
ADD B
JNC L1.
INR C
L1.: STA 4152
MOV A,C
STA 4153
HLT
```

---

③ Subtraction

```
MVI C,00
LDA 2500
MOV B,A
LDA 2501
SUB B
JNC L1
CMA → compliment A content
INR A → Increment A
INR C
L1: STA 2502
MOV A,C
STA 2503
HLT.
```

---

④. Multiplication of 2 8-bit

```
LDA 2500
MOV B,A
LDA 2501
MOV C,A
MVI A,00 → counter
MVI D,00 → carry (with carry)
L1: ADD B
    JNC L2.
    INR D
L2: DCR C
    JNZ L1.
    STA 2502.
    MOV A,D
    STA 2503
    HLT.
```

---

⑤. Division of 2 8-bit

```
LDA 2500
MOV B,A
LDA 2501
MVI C,00
LOOP1: CMP B
    JC LOOP
    SUB B
    INR C
LOOP: STA 2502
    MOV A,C
    STA 2503
    HLT.
```

```
LOOP : STA 2502
       MOV A,C
       STA 2503
       HLT
```

---

⑥ Smallest no. in an array.

```
       LXI H, 4200  - count size
       MOV B,M      → count
       INX H
       MOV A,M
       DCR B
LOOP:  INX H
       CMP M → if A-reg <M go to loop 1
       JC loop1
       MOV A,M → set the new no as smallest
loop1: DCR B
       JNZ LOOP → Repeat comparison till B=0
       STA 4300 → store smallest value at 4300.
```

---

⑦ Largest no. in an array.

```
       LXI H 2000
       MOV B, M
       INX H
       MOV A,M
       DCR B
LOOP   INX H
       CMP M
       JNC loop1
       MOV A,M
loop1: DCR B
       JNZ LOOP
       STA 4300
       HLT.
```

```
       LXI H, 2500
       MOV C,M
       INX H
       MOV A,M
       DCR C
L1:    INX H
       CMP M.
       JNC L2.
       MOV A,M
L2:    DCR C
       JNZ L1
       STA 2400H
       HLT.
```

Ascending.

```
       MVI B, COUNT H
       DCR B
L3:    MOV C,B
       LXI H, 2501H
L2:    MOV A,M.
       INX
       MOV D,M
       CMP M.
       JC L1.  ~ JNC ⇒ Descending
       MOV M,A
       DCX H
       MOV M,D
       INX H
L1:    DCR C
       JNZ L2
       DCR B
       JNZ L3
       HLT.
```

DAA

LSB > 9 or AC = 1

⇒ 6 + LSB

MSB > 9 or CY = 1

⇒ 6 + MSB

Interrupt

→ It is a signal send by an external device to the MP to perform a task or work.

Process

- MP always check for interrupt at last M/C.
- If any → accept & sent INTA signal.
- Current value of PC saved → SP
- Starting addr. of ISR will copied to PC.
- MP execute ISR.
- Return back to main pgm when RET is called & retrieve back starting addrs from SP.

Classification

1. Maskable & Non-maskable
2. Vectored & Non-vectored.
3. ~~Hardware & S/W~~

1. Maskable

- Interrupt that can be disabled or ignored by the inst. of CPU
- Masking prevents interrupt from disturbing main pgm execution.
- Usefull in case of Low priority interrupts.
- Eg: RST 7.5, 6.5, 5.5 & INTR.

2. Non-Maskable

- Interrupt that cannot be disabled or ignored by the inst. of CPU.
- Used in critical power failure. situations
- Eg: TRAP.

3. Vectored

- They have fixed vector addr., which is the starting addr. of the subroutine
- TRAP, RST 7.5, 6.5, 5.5,

4. Non-vectored.

- They don't have a predefined vector addrs.

Eg: INTR.

Types of Interrupts

1. S/W interrupt 2. H/W interrupt

- They are pgm inst.
- Inst. inserted at desired location in program.
- While running pgm, if S/W interrupt encountered, then the MP execute ISR.
- 8 - S/W interrupts → RST0 → RST7

2. H/W interrupt

- External device initiate the H/W interrupt.
- By sending an appropriate signal to MP through interrupt pin.
- 5 - H/W interrupt

TRAP RST 7.5, 6.5, 5.5 INTR.

a) TRAP

- Non-maskable interrupt
- Vectored interrupt
- Highest priority
- edge & level triggered

b. RST 7.5

- maskable Controlled by DI & EI.
- Vectored
- 2nd Highest
- edge sensitive.

c) RST 6.5, 5.5

- maskable DI & EI.
- Vectored.
- 3rd & 4th priority
- Edge & level triggered.

d) INTR

- Maskable DI & EI.
- Non-vectored.
- Lowest priority
- level sensitive

Masking / unmasking of Interrupt

- EI, DI, SIM, RIM.

→ Inst sets the interrupt enable FF.

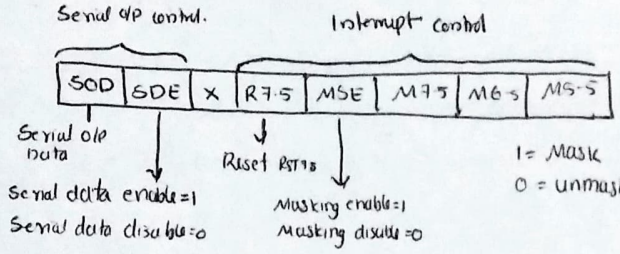
- RST 7.5, 6.5, 5.5 & INTR → enabled using EI

DI

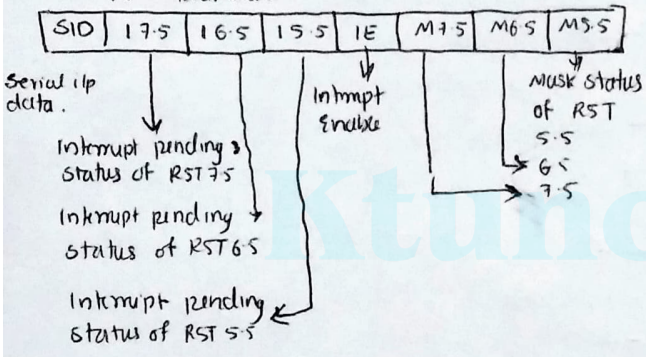
→ Reset the interrupt enable FF.

- RST 7.5, 6.5, 5.5

SIM. : Set Interrupt Mask.

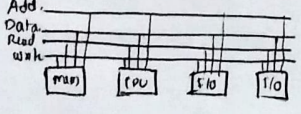


RIM - The status of pending interrupts can be read from Accumulator



I/O interfacing

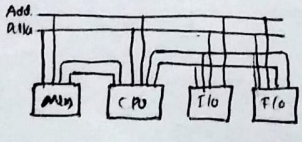
- Interfacing is the process of connecting external devices with µp so that they can exchange information.
- Keyboard, printer.



I/O interfacing Technique.

→ Memory maped I/O

→ I/O maped I/O.



memory maped I/O	I/O maped I/O.
1. single addr. space for memory & I/O devices.	• memory & I/O devices are addrs. separately.
2. uses same instructions to access both mem / I/O devices	• Use diff. instns for mem & I/O device. IN, OUT, I/O

for both memo & I/O devices are used → memo & I/O devices

- LDA, STA, LDAXrp STAXrp, • IN & OUT.
  etc.
- Data transfer blw any regis & I/O devices
- blw accu & I/O devices
- Device addr. is 16 bit $A_0 - A_{15}$
- 8- bit addrs $A_0 - A_7$ or $A_8 + A_{15}$

Memory Interfacing

- process of connecting memory with µp through buses & other h/w

1. **Address Decoding Techniques**

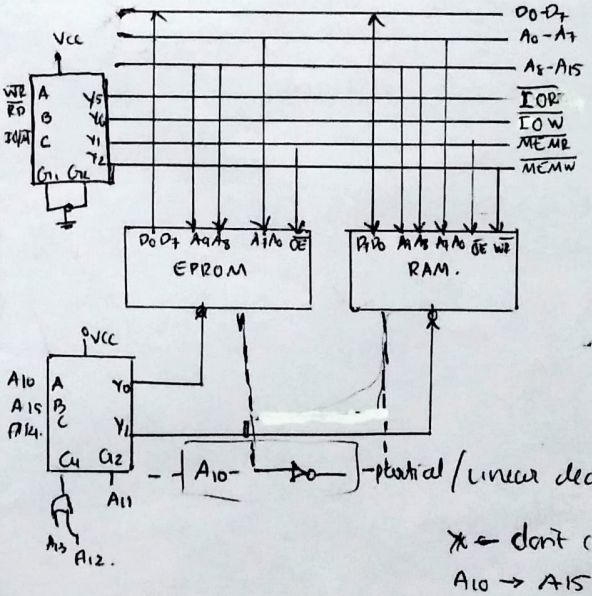
→ Absolute decoding (Full decoding)

→ Linear decoding (partial decoding)

Absolute decoding

- All available addr. lines are used for decoding to select a location.

Linear decoding



linear.

1. All higher address bit lines are used for decoding to select the mem or I/O device	1. Few higher add. are used for decoding to select the mem. or IO device.
2. More H/w is required to design decoding logic	2. less H/w is required & sometimes it can be eliminated
3. High cost for decoding ckt	3. low cost
4. No multiple add	4. has multiple address
5. Used in large s/m	5. used in small s/m.

Interfacing I/O Port.

- Programmable peripheral Interface PPI.
- 8255. is a general purpose I/O device that designed to transfer I/O to µP.

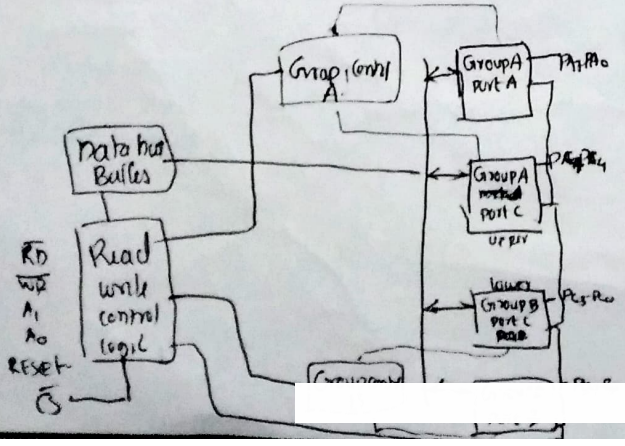
- Features
  - 3 8-bit IO ports. PA, PB, PC
  - Add/Data bus must be externally demuxed
  - TTL compatible.
  - It has improved DC driving capability

Functions of pin

1. Data bus (D_0-D_7) - 8 bit - Bidirectional.
2. CS - low signal → data trans
3. $\overline{RD}$
4. $\overline{WR}$
5. Address (A_0-A_1)

A_1	A_0	
0	0	PA
0	1	PB
1	0	PC
1	1	control reg.

6. RESET



7. Data Bus Buffer
8. Read/write control logic ← $\overline{RD}$ $\overline{WR}$ A_0 A_1
9. Group A & Group B control
10. port A, B, C. → 8-bit Buffer I/O latch
    → 8-bit unlatched buffer

Modes of operation

→ Bit set Reset (BSR) mode.
→ I/O mode

1. BSR mode

- any one of 8-bit of port C can be set or reset depending upon the select bits on control word register

2. I/O mode.

Mode 1 (Strobed I/O mode)

- port A & B → i/p or o/p
- handshake signals are exchanged
- i/p & o/p data are latched.

Mode-0 (Simple I/O)

- port A & B & C.
- Simple i/p or o/p.
- any port can be used as an i/p or o/p.
- Don't have Handshake / interrupt capability

Mode-2 (Strobed bi directional)

- port A.
- Allows bidirectional data transfer
- using handshake

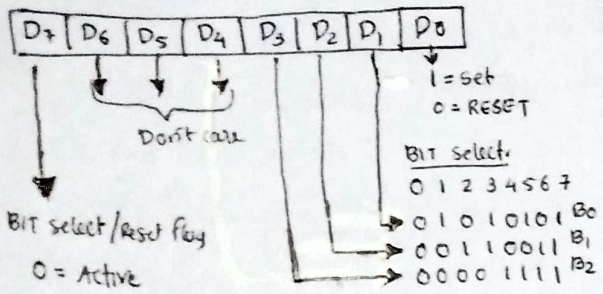
Handshaking Signals

- dedicated signals to coordinate data transfer between two devices with different speed.

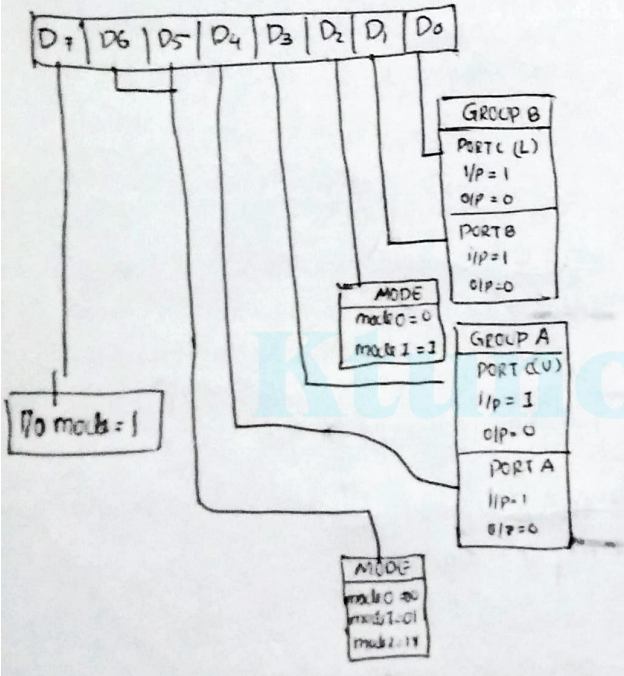
eg:

Control word format.

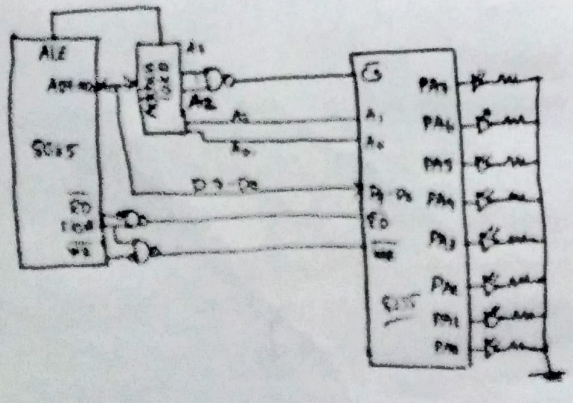
BSR mode.



I/O mode



Interfacing of 7 segment display



D7 D6 D5 D4 D3 D2 D1 D0
1 0 0 0 0 0 0 0

Program → to blink continously

```
MVI A 80 → initialize 8255
OUT CWR
BACK: MVI A, FF → To send 1 to all LED's at port A.
      OUT PORTA
      CALL DELAY
      MVI A,00 → To send 0 to all LED's at port A
      OUT PORTA
      CALL DELAY
      JMP BACK.
```

Delay :

```
DELAY: MVI C, FF
REP: DCR C
     JNZ REP
     RET.
```

Program- display digits 0-9.

```
MVI A,FF
OUT C.WR
AGAIN: MVI A,3FH
       CALL DELAY ← OUT PORTA } 0
       MVI A,06H
       OUT PORTA
       CALL DELAY } 1
       MVI A,5B
       OUT PORTA
       CALL DELAY } 2.
       :
       JMP AGAIN
```

```
DELAY: MVI C, FFH
BACK: DCR C
      NOP
      NOP
      JNZ BACK
      RET.
```

I/O PORTS

Port 0

- P.00 – P0.7.
- normal bidirectional I/O port - or
- used for add/data interfacing for accessing external mem.
- When control = 0 ⇒ bidirectional I/O port
  - 1 ⇒ Add/data interface

Port 0 → i/p

- control = 0
- '1' is written to the latch → Both MOSFET are off.
- Hence o/p pin has floats - data on pin is directly read by read pin.

Port 0 → o/p

- control = 0

Port -1

- 8 pins – P1.0 → P1.7.
- Dont have alternate function
- dedicated only → I/O interfacing

Output port

- The pin is pulled up or down through internal pull-up

i/p port

- '1' has to be written to the latch.
- Bit written to the pin by external device can be read by processor.

Port -2

- 8 pins – P2.0 → P2.7.
- used for higher external addr byte or normal I/O port
- port-2 latch remains stable when port-2 pin are used for external mem access

Port -3

- 8 pins - P3.0 → P3.7
- Each pin have alternate function.
- can individually pgmd for I/O func or alternate func.
- Alternate function active only when latch = 1.
- Alternate functions are

RxD
TxD
$\overline{INT0}$
$\overline{INT1}$
T0
T1
$\overline{WR}$
$\overline{RD}$

Register organisation

- 256 bytes

- 128 → 00h – 1Fh – Register banks
  → 20h – 2Fh – Bit add RAM
  → 30 – 7Fh – General purp RAM
  } Internal RAM

128 → 80h → FFh – SFR.

Internal RAM

1. Register banks
2. Bit addressable RAM
3. General purpose RAM.

1. Register Banks

- 32 working regis combined to form 4 banks of 8 registers each.

- ~~R0, R1, R2, R3~~
- 4 registers banks are numbered 0-3 made up of registers named $R_0 - R_7$
- RS0 & RS1 in PSW determins which bank is using
- Bank 0 is selected when on Reset

2. Bit addressable RAM

- 16 bytes are bit addressable
- 20H → 2FH
- 16 bytes provide 128 bits of RAM bit
- Bit variable can set and cleared with comomand

  SETB 25h

  CLR 25h.

  CPL 25h compliment

- Direct addres mode → Single bit inst.

3. General purpose RAM.

- 30h → 7Fh
- addressable as byte.
- 80 bytes of internal RAM are available.
- Direct or indirect addressing

  MOV A, 6Ah

4 SFR

- 80 - OFFh.
- Some SFR are bit addressable.
- 21 registers

  ACC, B, PSW, SP, DPH, DPL, P0, P1, P2, P3

  IP, IE TMOD, TCON, TH0, TL0 TH1, TL1

  SCON SBUF PCON.

PSW.

CY	AC	F0	RS1	RS0	OV	-	P

TMOD.

GATE	C/T	M1	M0	GATE	C/T	M1	M0

~~TCON~~

M_1	M_0	mode
0	0	0 - 13 bit Timer
0	1	1 - 16 bit timer
1	0	2 - 8 bit Auto-Reload T/C
1	0	3 - (Timer 0) TL0 - 8 bit T/C controlled by T0 control bits - TH0 - 8 bit Timer controlled by T1 control bits
1	1	3 - (Timer 1) T/C 1 Stopped.

TCON.

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TF - Timer flow

TR - Timer run

IE - External Interrupt

IT - Interrupt type.

SCON.

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

SM - Serial port mode

REN - Receiver enable bit.

TB8 - Transmit 9th bit

RB8 - Receiver bit(9th)

TI - Transmit interrupt flag

RI - Receive interrupt flag

PCON

SMOD	-	-	-	GF1	GF0	PD	IDL

PD - power clown

IDL - Idle mode bit

Bit handling inst

SETB C

CPL C

CLR C

Bit manipulation inst

CLR C

CPL C

SETB C

ANL C

ORL C

MOV C

JC.

DJNZ

Assembler directives

- They are spl inst to the assemble pgm and are used to define specific operations
- part of executable pgms
- ORG - indicate the beginning of address
- EQU - used to define a constant without occupying a memory loca

→ COUNT EQU 25

MOV R3, #COUNT

- DB - used to define a 8-bit data.

→ DATA DB 34.

- DW - define word
- DBIT - define Bit
- END - last line of an 8051 pgm, to stop the assembler pgm.

DELAY ROUTINES

500 MS delay.

MOV R5, #250D.

LOOP : DJNZ R5, LOOP1

RET.

1 ms delay

MOV R5, #250D

MOV R6, #250D

LOOP1: DJNZ R5, LOOP1

LOOP2 : DJNZ R6, LOOP2

RET.

1 Sec. delay

MOV R5, #250D.

LOOP : ACALL DELAY

ACALL DELAY

ACALL DELAY

ACALL DELAY

DJNZ R5, LOOP.

DELAY : MOV R6, #250D

MOV R7, #250D

L1 : DJNZ R6, L1

L2 : DJNZ R7, L2

RET.

PORT 0

output

MOV A, #00H

BACK: MOV P0, A

ACALL DELAY

CPL A

SJMP BACK.

input

MOV A, #0FFH

MOV P0, A

BACK: MOV A, P0

MOV P1, A

SJMP BACK.

PORT 1.

output.

```
    MOV A,#00H
BACK: MOV P1, A
      ACALL DELAY
      CPLA
      SJMP BACK.
```

input

```
MOV A, #0FFH
MOV P1, A
MOV A, P1
MOV R7, A.
```

PORT-2

output

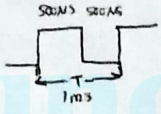
```
    MOV A, #00H
BACK: MOV P2, A
      ACALL DELAY
      CPL A
      SJMP BACK
```

input.

```
MOV A,#0FFH
MOV P2, A
MOV A, P2
MOV R7, A.
```

Program for generating 1KHz. Square wave.

$$T = \frac{1}{f} = \frac{1}{1 \times 10^3} = 1 \text{ msec.}$$




```
      ORG 000H.
      MOV A, #00H
BACK: MOV P, A
      ACALL DELAY
      CPL A
      SJMP BACK
      END.
```

```
DELAY: MOV R5, #250
       MOV R6, #250
LOOP:  DJNZ R5, LOOP
       RET.
```

1. find the MC for 11.0592 MHz

$\Rightarrow \frac{11.0592}{12} = 921.6\ kHz$

machine cycle = $\frac{1}{921.6}$

$= 1.085\ \mu s$

---

2. find the size of the delay

```
    MOV A, #55H
A:  MOV P1, A
    ACALL DELAY          1µs = 1 MC
    CPL A
    SJMP A
```

time delay

```
DELAY: MOV R3, #200      1
Here : DJNZ Here         200x2
       RET               2
```

$(1+(200\times2)+2) \times 1.085\ \mu s = 436.255\ \mu s$

---

③

```
DELAY: MOV R3, #250      1
HERE:  NOP               1   (250 x 6) 1.85
       NOP               1   = 1630.755 µs
       NOP               1
       NOP               1
       DJNZ : HERE       2
       RET               2
```

---

④ Square wave of 50% duty cycle.

- 50% → ON & OFF → same length.
- P1.0 → toggle with same time delay

```
    HERE SETB P1.0
         LCALL DELAY
         CLR P1.0
         LCALL DELAY
         SJMP HERE
```

(b)

```
HERE:
    CPL P1.0
    LCALL DELAY
    SJMP HERE
```

DATA types

1. unsigned chara
2. Signed chara
3. unsigned int
4. Signed int
5. Sbit (single bit)
- Bit & sfr.

1. unsigned character.

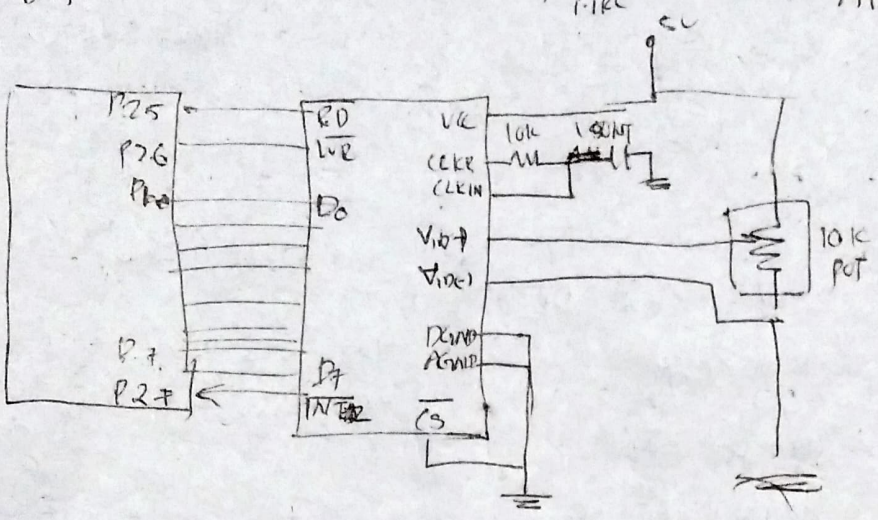
- 8 bit data type 0-255
- most widely used.
- Counter value
- Ascii chara.

Write an 8051 C pgm to send values 00-ff, to port 1.

```
⇒ #include <reg51.h>
  void main (void)
  {
    unsigned char z;
    for (z=0; z<=255; z++)
      P1 = z
  }
```

DAC.

ADC 0804



MOV P1, #0FFH.

CLR P2.6

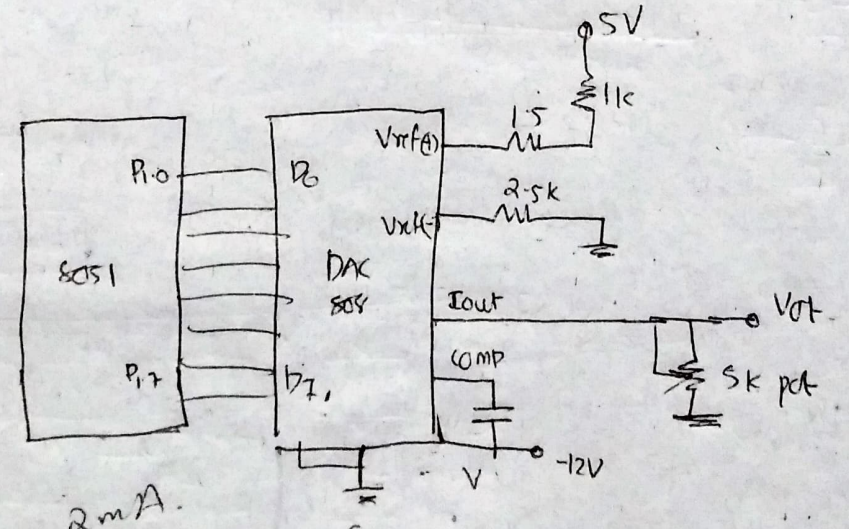
SETB P2.6

Here JB P2.7, Here

DAC 808

CLR P2.5

MOV A, P1



2mA.

$$I_{out} = I_{ref}\left(\frac{D7}{2} + \frac{D6}{4} + \frac{D5}{8} + \frac{D4}{16} + \frac{D3}{32} + \frac{D2}{64} + \frac{D1}{128} + \frac{D0}{D4} \right.$$

PSW

8085

S	Z	X	AC	X	P	X	CY

8051

CY	AC	F0	RS1	RS0	OV	-	P

RS1	RS0	
0	0	Rb0
0	1	Rb1
1	0	Rb2
1	1	Rb3

1024 bytes memory

74LS138

74LS373